Homework 1

Sam Kutsyn, 2581500 EE 446

July 23, 2026

Prelude

See submission files in ./submission folder.

Problem 1

Part (e) Implementation Summary

For Part (e), I combined Knowledge Distillation, Pruning, and INT8 Quantization to achieve maximum model compression. I started with the smaller student architecture from Part (d) and trained it using distillation loss to retain the teacher model's accuracy. During this training, I applied magnitude-based weight pruning (using a PolynomialDecay schedule) to force 80% of the student's parameters to zero. Finally, I stripped the pruning wrappers and applied full INT8 quantization during the TFLite conversion. By stacking these three techniques I achieved a significantly smaller model footprint than any single compression method could provide on its own, while maintaining strong classification performance.

Model File	Compression Technique Applied	Size (KB)
`model_base.tflite`	None	14.07
`model_float16.tflite`	Float16	8.95
`model_pruned.tflite`	Pruning + DRC	8.24
`model_dynamic.tflite`	DRC	8.17
`model_kd.tflite`	Knowledge Distillation + DRC	6.11
`model_int8.tflite`	INT8 Quantization	5.74
`model_kd_prune_int8.tflite`	KD + Pruning + INT8 Quantization	3.64

Problem 2: Exploring Edge Impulse

Learning Blocks in Free-Tier Edge Impulse

Classification Predicts discrete categories.

Example Model: A Deep Neural Network (Dense/CNN) for Image Classification (e.g., classifying images of cats vs. dogs) or Audio Classification (e.g., keyword spotting for "yes" and "no").

Regression Predicts continuous numerical values.

Example Model: A Deep Neural Network for Time-Series Regression (e.g., predicting the remaining useful life of a bearing based on vibration data, or estimating blood pressure from PPG signals).

Transfer Learning (Images) Uses pre-trained models adapted to a new task.

Example Model: MobileNetV1 or V2 (Image) / YOLOv5 (Object Detection). For instance, taking a pre-trained image recognition model and fine-tuning the final layers to recognize specific industrial defects.

Anomaly Detection Anomaly Detection (Clustering): Learns the "normal" state of data without labels and flags deviations.

Example Model: K-Means clustering or Gaussian Mixture Models (GMM) used for predictive maintenance (e.g., detecting abnormal vibration patterns in a machine).

Visual Anomaly Detection - FOMO-AD Detect visual anomalies. Extracts visual features using a pre-trained backbone, and applies a scoring function to evaluate how anomalous a sample is by comparing the extracted features to the learned model. Does not require anomalous data.

Layers under the "Classifier" section

When building a custom neural network architecture in the "Classifier" (Keras) block within Edge Impulse, the following standard layer types are available in the visual editor:

1. Dense: A standard neural network layer where every neuron is connected to every neuron in the previous layer. Used for final classification/regression reasoning and learning non-linear combinations of extracted features.
2. Dropout: A regularization layer that randomly sets a fraction of input units to zero during training. Purpose: Prevents overfitting by ensuring the network doesn't rely too heavily on any single neuron.
3. Flatten: Flattens a multi-dimensional input tensor into a 1D array. Purpose: Acts as a bridge between convolutional/pooling layers (which output 2D/3D feature maps) and Dense layers (which require 1D input).
4. Conv1D / Conv2D (Convolutional): Applies sliding filters over the input data. Purpose: Extracts spatial or temporal features. Conv2D is typically used for images, while Conv1D is used for time-series or audio data.
5. Reshape: Turn one-dimensional data from a DSP block into multi-dimensional data. Use this as an input to a convolutional layer. Use this for deep learning on raw data, or to process MFCC output.

Model Compression Options under "Deployment"

Under the Deployment section, Edge Impulse allows you to optimize and compress your model before exporting it to C++ code. The available options include:

Unoptimized (float32) The baseline model with no compression applied. Highest accuracy, but largest memory footprint.

Quantized (int8) Converts the model's 32-bit floating-point weights and activations to 8-bit integers. Purpose: Reduces model size by ~4x and significantly speeds up inference on edge hardware with minimal accuracy loss.

Synthetic Data Generation

In the free-tier version of Edge Impulse, synthetic data generation is primarily supported for two input modalities:

Image Data Edge Impulse integrates diffusion-based Generative AI models (similar to Stable Diffusion) and background augmentation pipelines. Users can generate synthetic images using text prompts or blend existing objects into new background contexts to simulate different environments and lighting conditions.

Audio Data Uses generative models (like AudioLDM or similar text-to-audio diffusion architectures) to generate environmental sounds or spoken words from text prompts. It also includes algorithmic data augmentation techniques (like adding background noise, pitch shifting, and time stretching) to existing audio samples.

Synthetic data is critical for overcoming data scarcity and class imbalance by generating realistic examples of rare events or anomalies that are difficult to capture in the real world. It also enhances model robustness by simulating diverse edge cases and environmental conditions and protects privacy by not using sensitive, real-world information.